



SAKARYA
ÜNİVERSİTESİ

T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
SİSTEM PROGRAMLAMA

TARSAU ARŞİVLEME PROGRAMI

Melike Türkdal

B241210355

2B

KULLANILAN KÜTÜPHANELER

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/stat.h>
5  #include <sys/types.h>
6  #include <unistd.h>
```

- **#include <stdio.h>:** Standart giriş/çıkış operasyonları (fopen, fclose, fprintf, fgetc) için dahil edilmiştir.
 - **#include <stdlib.h>:** Dinamik bellek yönetimi (malloc, free) ve programdan çıkış işlemleri (exit) için kullanılmıştır.
 - **#include <string.h>:** String karşılaştırma (strcmp), ekleme (strcat), kopyalama (strcpy, strdup) ve parçalama (strtok) fonksiyonları için eklenmiştir.
 - **#include <sys/stat.h>:** Giriş dosyalarının izin (permission) ve boyut (size) bilgilerini okumak (stat) amacıyla dahil edilmiştir.
 - **#include <sys/types.h>:** Sistem veri tiplerini ve izin işlemlerini desteklemek için kullanılır.
 - **#include <unistd.h>:** POSIX işletim sistemi API'sine erişim (dosya ve izin kontrol operasyonları) sağlamak amacıyla eklenmiştir.
- graf metniniz

FONKSİYONLARIN AÇIKLAMALARI

```
14 long check_file_suitability(const char *filename) {
15     FILE *file = fopen(filename, "r");
16     if (file == NULL) {
17         printf("%s giris dosyasi bulunamadi veya acilamadi!\n", filename);
18         exit(1);
19     }
20     long size = 0;
21     int ch;
22     while ((ch = fgetc(file)) != EOF) {
23         size++;
24         if (ch < 0 || ch > 127) {
25             printf("%s giris dosyasinin formati uyumsuzdur!\n", filename);
26             fclose(file);
27             exit(0);
28         }
29     }
30     fclose(file);
31     return size;
32 }
```

long check_file_suitability(const char *filename)

Çalışma Mantığı: Hedef dosyayı karakter karakter (fgetc) okur. Her karakterin sayısal değerinin standart ASCII tablosu sınırlarında (0 ile 127 arasında) olup olmadığını kontrol eder. Eğer bu aralığın dışında (Türkçe karakter veya binary/ikili veri gibi) bir değer tespit edilirse, dinamik olarak "[Dosya_Adı] giris dosyasinin formati uyumsuzdur!" mesajını yazar ve programı güvenli bir şekilde sonlandırır. Dosya tamamen uyumluysa toplam boyutunu bayt cinsinden döndürür.

```

34 void create_archive(const char *output_filename, char *input_files[], int file_count) {
35     char header[5000] = "";
36     char temp[256];
37
38     for (int i = 0; i < file_count; i++) {
39         struct stat st;
40         if (stat(input_files[i], &st) != 0) {
41             printf("Hata: Dosya bilgileri okunamadi: %s\n", input_files[i]);
42             exit(1);
43         }
44         int permissions = st.st_mode & 0777;
45         long size = st.st_size;
46
47         sprintf(temp, "%s,%s,%ld", input_files[i], permissions, size);
48         strcat(header, temp);
49
50         if (i < file_count - 1) {
51             strcat(header, " ");
52         }
53     }
54
55     int header_bytes_size = 10 + strlen(header);
56     FILE *archive = fopen(output_filename, "w");
57     if (archive == NULL) {
58         printf("Hata: Arşiv dosyası oluşturulamadı!\n");
59         exit(1);
60     }
61
62     fprintf(archive, "%010d", header_bytes_size);
63     fprintf(archive, "%s", header);
64
65     for (int i = 0; i < file_count; i++) {
66         FILE *infile = fopen(input_files[i], "r");
67         if (infile == NULL) continue;
68         int ch;
69         while ((ch = fgetc(infile)) != EOF) {
70             fputc(ch, archive);
71         }
72         fclose(infile);
73     }
74     fclose(archive);
75     printf("Dosyalar birleştirildi.\n");
76 }

```

void create_archive(const char *output_filename, char *input_files[], int file_count)

Çalışma Mantığı:

1. Organizasyon Bölümü Yapılandırması: Bir döngü yardımıyla her bir giriş dosyasının stat sistem çağrısı üzerinden Linux izin kodları (st.st_mode & 0777 ile oktal biçimde) ve boyut verileri çekilir. Bilgiler |Dosya_adı,izinler,boyut formatına getirilerek aralarında birer boşluk olacak şekilde header stringinde birleştirilir.
2. Boyut Bilgisinin Yazılması: Oluşturulan organizasyon bölümünün toplam karakter boyutu hesaplanır. Dosyanın en başına sabit 10 bayt yer kaplayacak şekilde ve solu sıfırlarla doldurularak (%010d formatıyla) yazılır. Hemen ardına organizasyon metni eklenir.
3. Ham Verilerin Eklenmesi: Tüm giriş dosyaları sırayla tekrar açılarak içerikleri (ham ASCII verileri) hiçbir ayırıcı karakter kullanılmadan doğrudan arşiv dosyasının sonuna peş peşe yazılır.

```

78 void extract_archive(const char *archive_name, const char *target_dir) {
79     FILE *archive = fopen(archive_name, "r");
80     if (archive == NULL) {
81         printf("Arşiv dosyası uygunsz bozuk!\n");
82         exit(1);
83     }
84
85     char size_str[11];
86     if (fread(size_str, 1, 10, archive) != 10) {
87         printf("Arşiv dosyası uygunsz bozuk!\n");
88         fclose(archive);
89         exit(1);
90     }
91     size_str[10] = '\0';
92     int header_size = atoi(size_str);
93
94     if (header_size <= 10) {
95         printf("Arşiv dosyası uygunsz bozuk!\n");
96         fclose(archive);
97         exit(1);
98     }
99
100     int header_content_size = header_size - 10;
101     char *header_content = malloc(header_content_size + 1);
102     fread(header_content, 1, header_content_size, archive);
103     header_content[header_content_size] = '\0';
104
105     if (strcmp(target_dir, ".") != 0) {
106         mkdir(target_dir, 0777);
107     }
108
109     char *header_copy = strdup(header_content);
110     char *file_record = strtok(header_copy, " ");
111
112     char file_names[32][256];
113     int file_modes[32];
114     long file_sizes[32];
115     int extracted_file_count = 0;
116
117     while (file_record != NULL && extracted_file_count < 32) {
118         if (file_record[0] == '|') {
119             file_record++;
120         }
121
122         char name[256];
123         char mode_str[20];
124         long size;
125
126         sscanf(file_record, "%[^|],%[^|],%ld", name, mode_str, &size);

```

```

127         strcpy(file_names[extracted_file_count], name);
128         file_modes[extracted_file_count] = strtol(mode_str, NULL, 8);
129         file_sizes[extracted_file_count] = size;
130
131         extracted_file_count++;
132         file_record = strtok(NULL, " ");
133     }
134     free(header_copy);
135     free(header_content);
136
137     for (int i = 0; i < extracted_file_count; i++) {
138         char full_path[512];
139         if (strcmp(target_dir, ".") != 0) {
140             sprintf(full_path, "%s/%s", target_dir, file_names[i]);
141         } else {
142             sprintf(full_path, "%s", file_names[i]);
143         }
144
145         FILE *outfile = fopen(full_path, "w");
146         if (outfile == NULL) {
147             printf("Hata: %s dosyası oluşturulamadı!\n", full_path);
148             continue;
149         }
150         for (long j = 0; j < file_sizes[i]; j++) {
151             int ch = fgetc(archive);
152             if (ch != EOF) {
153                 fputc(ch, outfile);
154             }
155         }
156         fclose(outfile);
157
158         chmod(full_path, file_modes[i]);
159     }
160
161     fclose(archive);
162     if (strcmp(target_dir, ".") != 0) {
163         printf("%s dizininde dosyalar açıldı.\n", target_dir);
164     } else {
165         printf("Dosyalar geçerli dizinde açıldı.\n");
166     }
167 }
168

```

void extract_archive(const char *archive_name, const char *target_dir)

Çalışma Mantığı:

1. Header Okuma: Arşiv dosyasının ilk 10 baytını fread ile okuyup tam sayıya çevirir (atoi). Bu sayede organizasyon bölümünün sınırını öğrenir ve bu boyutta bir dinamik bellek alanı (malloc) ayırarak başlık verisini belleğe yükler.
2. Dizin Yönetimi: Kullanıcı geçerli izin (.) dışında mutlak veya göreceli bir hedef izin belirttiyse, mkdir(target_dir, 0777) sistem çağrısı ile ilgili klasörü otomatik oluşturur.
3. Metin Ayırıştırma (Parsing): strtok fonksiyonu vasıtasıyla boşluk karakterlerine göre her bir dosyanın kaydı ayrıştırılır. Ardından sscanf ile virgüllere bölünerek dosya adı, oktal izin kodu (strtol taban 8) ve boyut değerleri dizilere kaydedilir.
4. Dosyaları ve İzinleri Geri Yazma: Arşiv dosyasında kalındığı yerden başlanarak, ilgili dosyanın boyutu kadar karakter okunup yeni hedef dosyaya yazılır. Dosya kapatıldıktan hemen sonra chmod sistem çağrısı tetiklenerek dosyanın arşivlendiği andaki orijinal okuma/yazma/çalıştırma izinleri Linux işletim sistemine geri iletilir.

```

170 int main(int argc, char *argv[]) {
171     if (argc < 2) {
172         print_usage();
173         return 1;
174     }
175
176     if (strcmp(argv[1], "-b") == 0) {
177         if (argc < 3) {
178             printf("Hata: Arşivlenecek dosya belirtilmedi!\n");
179             return 1;
180         }
181
182         char *output_filename = "a.sau"; // Varsayılan değer
183         int file_count = 0;
184         char *input_files[32];
185
186         for (int i = 2; i < argc; i++) {
187             if (strcmp(argv[i], "-o") == 0) {
188                 if (i + 1 < argc && argv[i + 1][0] != '-') {
189                     output_filename = argv[i + 1]; //
190                     i++;
191                 } else {
192                     // -o'dan sonra veri yoksa veya başka parametre basılıyorsa varsayılanı kullan
193                     output_filename = "a.sau";
194                 }
195             } else {
196                 if (file_count < 32) { //
197                     input_files[file_count++] = argv[i];
198                 } else {
199                     printf("Hata: Giriş dosyası sayısı en fazla 32 olabilir.\n");
200                     return 1;
201                 }
202             }
203         }
204
205         long total_size = 0;
206         for (int i = 0; i < file_count; i++) {
207             long file_size = check_file_suitability(input_files[i]);
208             total_size += file_size;
209         }
210
211         long max_allowed_size = 200 * 1024 * 1024;
212         if (total_size > max_allowed_size) {
213             printf("Hata: Giriş dosyalarının toplam boyutu 200 MB'i geçemez!\n");
214             return 1;
215         }
216     }

```

```

216         create_archive(output_filename, input_files, file_count);
217     }
218     else if (strcmp(argv[1], "-a") == 0) {
219         if (argc < 3 || argc > 4) {
220             printf("Arşiv dosyası uygunsuz bozuk!\n");
221             return 1;
222         }
223         char *archive_name = argv[2];
224         char *target_dir = (argc == 4) ? argv[3] : ".";
225
226         int len = strlen(archive_name);
227         if (len < 5 || strcmp(&archive_name[len - 4], ".sau") != 0) {
228             printf("Arşiv dosyası uygunsuz bozuk!\n");
229             return 1;
230         }
231
232         extract_archive(archive_name, target_dir);
233     }
234     else {
235         print_usage();
236         return 1;
237     }
238     return 0;
239 }

```

int main(int argc, char *argv[])

- **Mod Ayrımı:** argv[1] parametresinin -b mi yoksa -a mı olduğuna bakılarak program ilgili fonksiyona yönlendirilir.
- **Dinamik Varsayılan Çıktı Yönetimi (-o Kontrolü):** Döngü esnasında -o parametresi aranır. Eğer -o hiç yazılmamışsa veya yazılıp arkası boş bırakılmışsa, ödev kuralına sadık kalınarak çıktı adı otomatik olarak "a.sau" olarak set edilir. Geçerli bir isim verilmişse o isim atanır.
- **Giriş Dosyası Sınırı (Maksimum 32):** Döngü esnasında toplanan dosya sayısı 32'yi geçtiği anda işlem durdurulur ve taşma (overflow) engellenerek hata mesajı basılır.
- **Boyut Sınırı (Maksimum 200 MB):** Arşivleme başlamadan önce tüm dosyaların boyutları toplanır ve toplamın 200 * 1024 * 1024 baytı geçip geçmediği doğrulanır.
- **Dosya Uzantı Güvencesi:** Arşiv açma modunda (-a), parametre olarak verilen dosyanın son 4 karakteri incelenerek .sau uzantısına sahip olup olmadığı ve parametre sayısının kurallara uygunluğu denetlenir. Aksi durumlarda "Arşiv dosyası uygunsuz bozuk!" çıktısı üretilir.

```
melik@MelikeTurkdal:~/sistem_programlama$ ./tarsau -b t{1..33}.txt -o
Hata: Giris dosyasi sayisi en fazla 32 olabilir.
```

Giriş dosya sayısı en fazla 32 olabilir koşulu sağlanmıştır.

```
melik@MelikeTurkdal:~/sistem_programlama$ ./tarsau -b t{1..32}.txt -o
Dosyalar birlesitirildi.
```

Arşiv dosyasının adı -o parametresinden sonra belirtilmediği için, varsayılan olarak a.sau kullanıldı.



```
a.sau
main.c
```

```
melik@MelikeTurkdal:~/sistem_programlama$ ./tarsau -a a.sau d1
d1 dizininde dosyalar acildi.
```



```
SISTE...
d1
t1.txt
t2.txt
t3.txt
t4.txt
t5.txt
t6.txt
t7.txt
t8.txt
t9.txt
t10.txt
t11.txt
t12.txt
t13.txt
t14.txt
```